

RL Course NPTEL - <https://nptel.ac.in/courses/106106143>

REF: <https://www.linkedin.com/in/sourab-m/> Assignments: **GREAT!**

<http://www.cse.iitm.ac.in/~ravi/courses/Reinforcement%20Learning.html>

<http://www.cse.iitm.ac.in/~ravi/courses/Reinforcement%20Learning.html>

W1 L5: Intro to ML (om)	5
W1 L6-RL Framework:	7
w1 L7 - Intro to Immediate RL (~30min)	14
w1 L8 - Bandit Optimalities (17min)	16
w1 L9 - Value Function Based Methods:	19
w2L10 - UCB1:	27
w2L11 Concentration Bounds:	29
w2 L12 UCB1 Theorem:	33
W2 L13 PAC Bounds	36
w2 L13 PAC Explanation and Naive Algo Proff (PAC Bounds)	49
w2 L14 PAC bound for MAB	59
ToDo (from 18min): w2 L14 Median Elimination:	63
ToDo - 18min	67
w2 L15 Thompson Sampling (13min):	70
w3L16 Policy Search:	73
w3L17 REINFORCE:	77
w3L18 Contextual Bandits:	87
w3L19 Full RL Introduction:	89
REINFORCE	92
w3L20 Returns, Value-functions and MDPs	95
w4L21 MDP Modeling (33min)	101
w4L22 Bellman Equation (14min)	105
w4L23 Bellman Optimality Equation (30min)	108
ToDo: w4L24 Cauchy Sequence and Greens Eq. (31min)	113

ToDo: w4L25 Banach FixedPoint Theorem (25min)	114
ToDo: w4L26 Convergence Proof (18min)	115
w5L27 L_{π} Convergence (just screen shots)	117
w5-L28 Value Iteration Proof (31min)	120
A and b are Banach Theorem	120
ToDo: Read writeup (to be put up)	123
w5-L29 Policy Iteration (13min)	124
w5-L30 DP (35min)	128
Ref: MDP - Putterman - dont' try to read	131
w5-L31 Monte Carlo (23min)	133
w5-L32 Control MC (28min)	138
w6-L33 Off-policy MC	145
w6-L34 UCT	149
w6-L35 TD(0)	154
Ch6 - Book - Temporal Difference Methods (Soul of RL)	154
w6-L36 TD(0) Control	160
w6 L37 Q-Learning (30min)	165
w6 L38 Afterstate (7min)	174
w7L39 Eligibility Traces	175
w7L40 Backward View Eligibility Traces	181
1st edition of book section 7.4	184
Dutch trace:	186
w7L41 Eligibility Trace Control	187
w7L42 Thompson Sampling Recap	194
w8L43 Function Approximation (38min)	197
w8L44 Linear Parameterization (15min)	201
w8L45 State Aggregation Methods (22min)	205
w8L46 Function Approximation and Eligibility Traces (27min)	211
w8L47 LSTD and LSTDQ (49min)	217
w8L48 LSPI-FittedQ (17min)	229

w9L49 DQN Fitted Q-Iteration	235
w9L50 Policy Gradient Approach	239
w9L51 Actor-Critic REINFORCE	245
ToDo: w9L52 Proof of Theorem I	249
w9L53 Policy Gradient with Function Approximation	251
w10L54 Hierarchical Reinforcement Learning	255
w10L55 Types of Optimality	259
2. Recursively Optimal	261
3. Flat Optimality	261
w10L56 Semi-MDP	263
REF: Survey paper : Recent advances in Hierarchical RL	265
w10L57 Options Framework	267
Reference:	267
w10L58 Learning with Option	270
Research - Options Discovery (REF)	273
w10L59 HAM - Hierarchical Abstract Machines	275
Ref: Read first 4 sections: Recent advances in Hierarchical RL	275
w11L60 MAXQ	279
REF: MAXQ - journal paper is ideal - how to write good paper	279
w11L61 MAXQ Value Function Decomposition (42min)	283
w11L62 Option Discovery (16min)	293
w12L63 POMDP Introduction	295
w12L64 Solving POMDP	300
Refer Video	305

Game Plan:

ToDo:

1. W2 L14 Median Elimination (ToDo from 18min)
2. w4L24 Cauchy Sequence and Greens Eq. (32min)
3. w4L25 Banach FixedPoint Theorem (26min)
4. w4L26 Convergence Proof (18min)
5. w9L52 Proof of Theorem I (NOT There)

W1 L5: Intro to ML (om)

Some interaction with system

Behavioural Pysocholgy conditioning:

Pawell's Dog experiment

Dog's experiment -> Salivating when sees

Bell - is signal

Food is reward

Sutton - Brain

Cycling is NOT supervised

1983 - adaptive neron -> kick started RL

Question w1A1-1:

rewards and punishments based on which you are learning can be fairly delayed in time they need not be temporally close to the thing that cost it,

10min:



What is Reinforcement Learning?

- Learning about stimuli and actions based on rewards and punishments alone.
- No detailed supervision available
- Trial-and-error learning
- Delayed rewards
- Sequence of actions required to obtain reward
- Associative learning required
 - Need to associate actions to states
- Learn about policies not just actions
- Typically in a stochastic world

Learning from REWARDS and PUNISHMENTS ONLY

1. NO supervised data is available

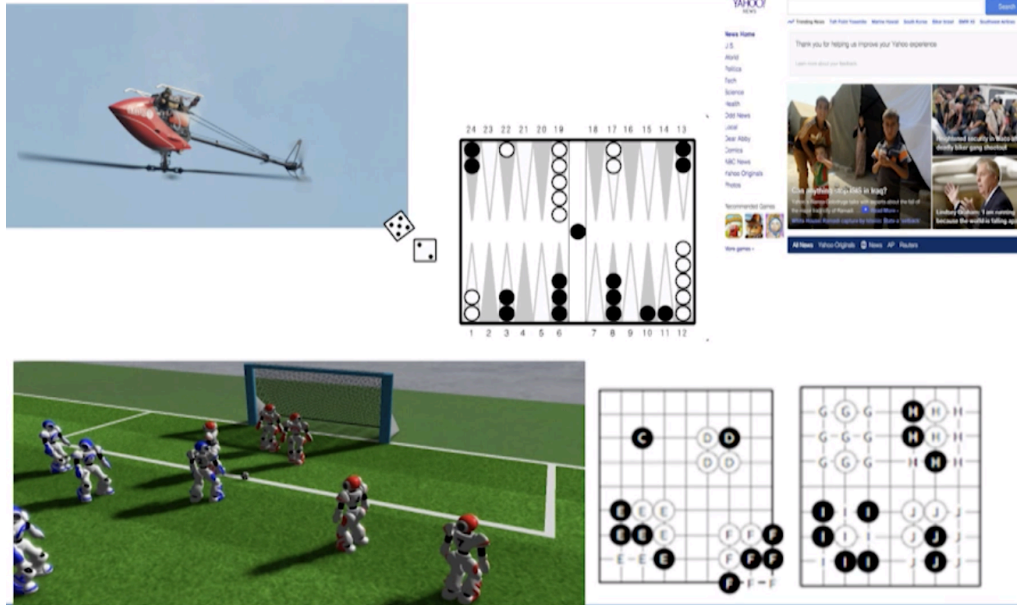
In chess, if you win you get reward. NOT for a move

Rewards fairly delayed in time -> drop a catch -> batsman scored 100+, it will cost you that much

When you ran over the stone, it is not that when you jumped and fell that caused injury to you

Chess - seq of action - you have to finish playing the game

Learn association. Inputs - state - how fast
 Learning a Policy to behave - place chase
 It happens in noisy, stochastic world
RL Applications:



Helicopter is flying upside down. Team stanford, Berkeley

Complex control machine - how it learnt it w/o human intervention

Bachmann - large branching factor

- Jerry Tassaro - Neuro Gammon

Used RL -> Played against itself - using Self Play

- TD gammon was better

Go: Branching Factor is Huge - Search Tree is very complex

- It is miracle Humans Play such a complex game

TD-Search

Balance Robot on leg - to kick

Top-Right: -> Makes Money -> **Solve Online Learning - no apriori feedback**

- Show news stories, of 20 stories which one to put prominently
- Can't have supervised

RL is natural way of solving problems

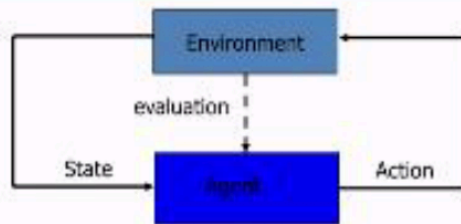
- How are ads selected
- Slate of ads - 10 ads should give right payoff

Computational Advertising:

27min: Andrew's video:

Helicopter video:

W1 L6-RL Framework:



- Learn from close interaction
- Stochastic environment
- Noisy delayed scalar evaluation
- Maximize a measure of long term performance

ENV - helicopter, cycle, bacgammon game, opponent

What action you take - makes env change state

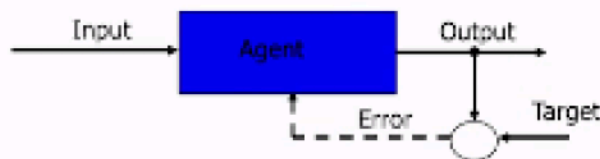
Behave Optimally - wining gives 100 point

- For each action you get 0 or nothing

Theory - evaluation is just mathematically

- State signal, evaluation is fiction
- Reward is a scalar signal -> convert to a number (getting hit -100, winning 100)
- Goal is to get as much of that quantity as long run
- Mathematically easier to optimize
- Which component of vector is optimal
- Argu - for and against - Scalar reward

Not Supervised Learning:



- Very sparse "supervision"
- No target output provided
- No error gradient information available
- Action chooses next state
- Explore to estimate gradient – Trail and error learning

Error gives direction -

You got 3 -> you don't know out of 100 or 10, so don't know if it is better

Have to change para to make it higher than 3

You max. Evaluation, don't reduce error

NOT Unsupervised:



- Sparse “supervision” available
- Pattern detection not primary goal

Primary goal: Want to detect pattern in the input, that is

12min: Temporal Difference Learning (Soul of RL):

- Simple rule to explain complex behaviors
- Intuition: Prediction of outcome at time $t+1$ is better than the prediction at time t . Hence use the later prediction to adjust the earlier prediction.
- Has had profound impact in behavioral psychology and neuroscience!

Dog was predicting outcome - if bell rings, will get food

Outcome is reward and punishment

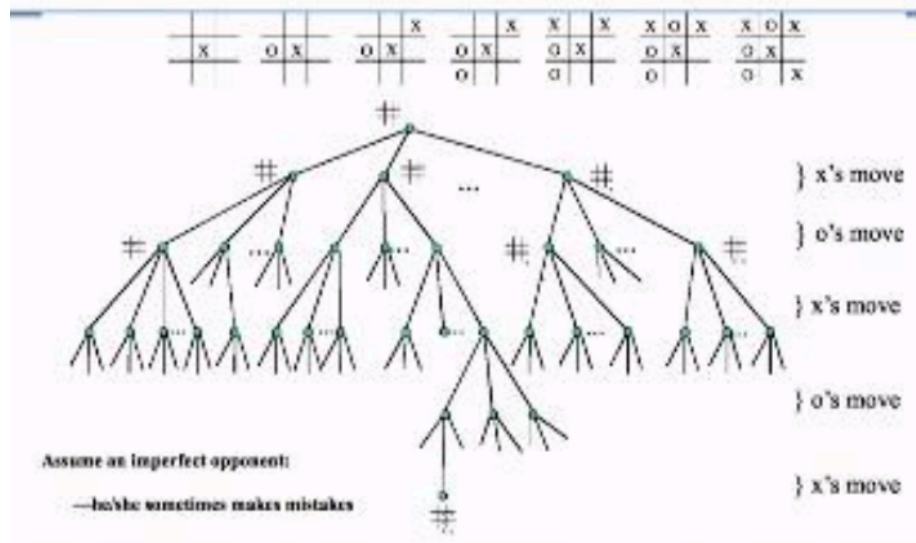
When I am closer to end of game

- I can confidently predict
- When you start chess - less confident, if have 5 pieces - more confident
- Prediction at $t+1$ is more confident than t
- Go forward, and check what prediction i will make, go back and modify prediction i make at time t . If it was 0.6, if i find that I will loose the game or worse move, change it to 0.55 (reduce it) in previous state

Widely accepted that Animals use TD

- Neuro transmitter behaviour in brain

Tic-Tac-Toe



Note is: Assume and imperfect opponent - he/she sometime makes mistakes

x -> 9 possible branches, for each 8 possible branches

How will you formulate it as an RL problem

- Move 9 actions - initially
- Reward - at the end of the move win - get 1, don't win get 0
- Keep playing it multiple times

Figure out where opponent makes mistakes

- Actions are wrt board position

The way we will solve as follows:

- For every board position play to till end and estimate
- Average over multiple games (in some games will win = 1, in some games not win)
- Prob of winning - expected reward starting from a position
- It can be a function if payoff is say 5. Or 1 for winning, -1 for not winning
- The expectation is well defined
- How will i decide whichever one has highest probability, put it there
 - In above example, which x move or o move
- What will be expected pay-off

23min: How do I learn this

Keep track of the trajectory - if you win - update trajectory -> increase probability of winning

- If not win - update trajectory -> increase probability of not winning
- Keep entire history - update the position

Drawback:

- Have to wait to till end
- Keep history

What i have played will be non-zero, others are zero

- Will be playing the same game again and again

Explore option

- I should not always be doing best moves
- Are there better moves than current i think is the best
- In greedy you will play fixed play

33min: Question:

When estimating prob of winning

- Say it is 0.8, i want to explore, it gives prob of 0.4 (down below)
- Should i use 0.4 to update 0.8?
- I have to change 0.4, do i need to change it to 0.8?

1. If I am playing to win I will take the best action. Will no penalise 0.4
- 2.

I am behaving now, it should be current policy so i should update it

Write answer to it

Two algorithms - one does option1, other does option2

- 1 wait till end
- 2 keep adapting as you go along

Have to explore in both cases

37min: Explore Exploit Dilemma

- One key question - the dilemma between exploration and exploitation
- Explore to find profitable actions
- Exploit to act according to the best observations already made
- *Bandit problems* encapsulate 'Explore vs Exploit'
- Chapter 2

When do know you have explored enough

It will ignore delayed rewards, sequence of action

- Goal is to pick action which gives highest rewards
- You don't know the prob distribution through which these rewards are coming
- Have to exploration - take every action at least - if it deterministic
- For stochastic - have to try multiple times

40min

2 classes of algo: 1 based on DP

Dynamic Programming:

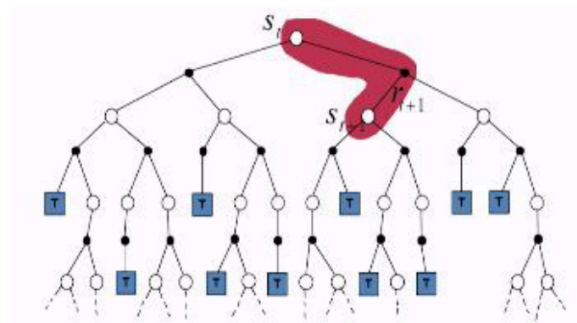
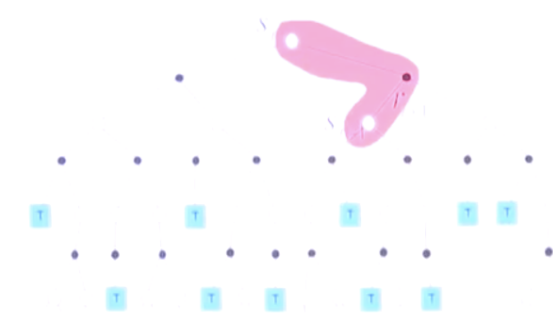
- DP is the solution method of choice for completely specified RL problems
 - Require complete knowledge of system dynamics
 - Expensive and often not practical
 - Curse of dimensionality
 - Guaranteed to converge!
- RL methods: online approximate dynamic programming
 - No knowledge of P and R
 - Sample trajectories through state space
 - Some theoretical convergence analysis available

Repeated sub structure,



Learn expectations for winning, if I am going to take $n-1$ steps left, you chose it for when n steps left

Simplest TD Method



Will learn - Online DP

Use only one step to update prob - instead of entire tree/out come - use samples

For TD what will you do - will go till the end.

When doing for first all will be zero

- Once you reach end - you update and it keep going up

44 Question:

- If I update at end - update from $s+1$, instead $s+1$ could be from any path
- Crucial difference
-

45 Solution Methods (Classes of Algorithm)

- Temporal Difference Methods
 - $TD(\lambda)$
 - Q-learning
 - SARSA
 - Actor-Critic
- Policy Search
 - Policy Gradient Methods
 - Evolutionary algorithms
- Stochastic Dynamic Programming

Policy Search will be covered from other material

Other Applications:

- | | |
|--|---|
| <ul style="list-style-type: none">• Optimal Control<ul style="list-style-type: none">– Robot Navigation– Chemical Plants• Combinatorial Optimization<ul style="list-style-type: none">– Elevator Dispatching– VLSI placement and routing– Job-shop scheduling– Routing algorithms– Call admission control• More<ul style="list-style-type: none">– Intelligent Tutoring Systems | <ul style="list-style-type: none">• Computational Neuroscience<ul style="list-style-type: none">– Primary mechanism of learning• Psychology<ul style="list-style-type: none">– Behavioral and operant conditioning– Decision making• Operations Research<ul style="list-style-type: none">– Approximate Dynamic Programming• More<ul style="list-style-type: none">– Dialogue systems |
|--|---|

Bio-tech people use RL a lot

Game Playing - Arcade Games:



Minh et al., Nature 2015



- Learnt to play from video input
 - from scratch
- Used a complex *neural network*!
 - Considered one of the hardest learning problems solved by a computer.
- More importantly *reproducible*!!

More game playing from Google - DeepMind's Atari Player (Video)

Albert - ALE (Arcade Learning Environment)

- Just by looking at pixels from screen

Reference: Only computational Learning paper to publish in Nature!

43 games better than human performance!

Currently in 48 games better than human

Next steps how humans process information

AMAZING - released the code!

- It is reproducible
- Reward was never given - only pixels were given!

Tom Mitchell

Chapter in Russell and Norvig

Neuro science - Sutton and Barto -> RL chapter

Mathem - Bertsekas and Tsitsiklis

MDP Puterman - to understand MDPs better

Sutton and Barto textbook the second edition

very short introduction to reinforcement learning in Tom Mitchell's book

chapter in Russell and Norvig and if you are interested in looking at the neuroscience perspective of reinforcement learning

the history of how behavioral psychology models evolved to temporal difference learning you can look at Dayan and Abbott it is one chapter on reinforcement learning you do not have to understand the neuroscience to understand the RL chapter.

So do not get put off by the title of the book right and then the more mathematically well grounded introduction to reinforcement learning would be from Bertsekas and Tsitsiklis so apart Markov decision processes by Puterman that will use certain chapters from Puterman right they are just for us to understand MDPs better markov decision processes better.

w1 L7 - Intro to Immediate RL (~30min)

Just for time t , take action a_t and get reward R_t

Give drug to an organism - which drug to administer

Whether the organism died or not

Outcome can be stochastic - may not get same evaluation back

- Sometimes i may succeed, sometimes may not
- So need to repeat choices multiple times
-

It introduces need for Exploration

- Sometime i want to stop and start giving the drug

If I know enough about the system

- Then start Exploiting

Explore - Exploit Dilemma

Address - Theoretical perspective - books says here is how you solve

- **Why do people solve the way they do - Intuition**

Set of action $A = \{ 1, \dots n \}$

Each action gets a payoff (reward, evaluation (optimisation), cost (control theory)- all denote same thing)

I will be using payoff -

When I select an arm (action)

- Will get sample from an unknown distribution
- Associated with each action - some prob of coming up head
- Head - payoff 1, tail 0
-

Action 1 -> flip coin 1, Action 5 -> flip coin 5

- Don't change which coin to flip based on Action

Will give reward based on outcome of the coin

- Payoff -> Prob of associated head is 0.8
- Expected payoff action 1 -> 0.8

Select an action (or arm) -> then toss coin -> depending upon the outcome of the coin -> reward

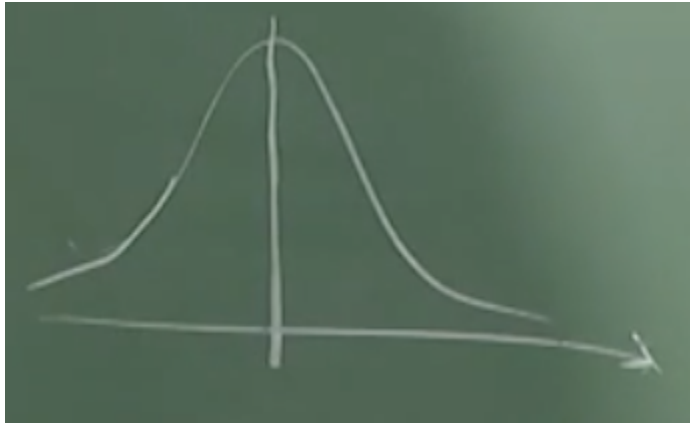
Remember when I say sample from distribution

When toss a coin - distribution is bernoulli, other distribution could be Gaussian distribution

X axis is payoff, y-axis you will see the payoff

19min: How you sample from Gaussian distribution:

- Generate random -> create cdf -> when cdf reaches



4 possible outcomes:

p_1, p_2, p_3, p_4

If no. is less p_1 than outcome 1,

between p_1 and p_1+p_2 , outcome 2

between p_1+p_2 to $p_1+p_2+p_3$, outcome 3

between $p_1+p_2+p_3$ and 1 outcome 4

In discrete add probability

Initially the distribution will be fixed

- Don't change coin
- Don't change Gaussian distribution

Pick action have highest mean for gaussian - pick that action

The challenge comes when I don't know the distribution

- Have to figure out the distribution

Assume for each action there is true expected payoff

$$q^*(a)$$

q^* of a - q^* is a function will give true expected payoff of action a

Coins p_1, p_2

Gaussian $q^*(1) \rightarrow \mu_1, q^*(2) \rightarrow \mu_2$

If I know q^* , I am done

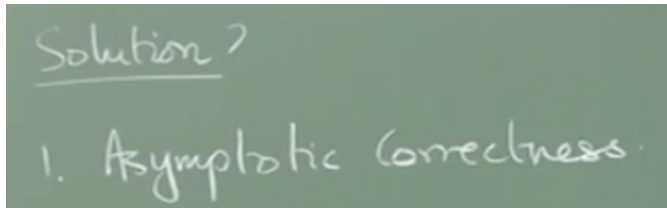
w1 L8 - Bandit Optimalities (17min)

Why is it called Bandit problem: One arm bandit is slot machine

- Put coin, pull lever, $\rightarrow$ then machine will steal your money
- One lever - one arm bandit
- N arms - every time to pull arm you have to pay money

Multi-arm bandit - similar to one arm bandit

You have to balance exploration/ exploitation



1. Asymptotic correctness

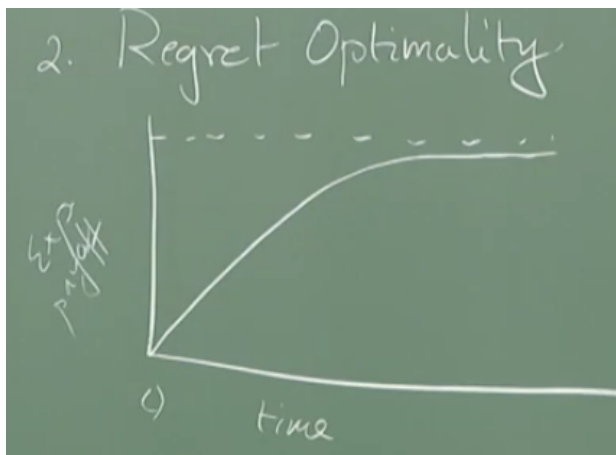
Don't put any bound on you

Give me a guarantee, eventually you will select the arm which has highest payoff

Come up with simple algorithm

2. Regret Optimality

If at time 0, knew which is optimal arm

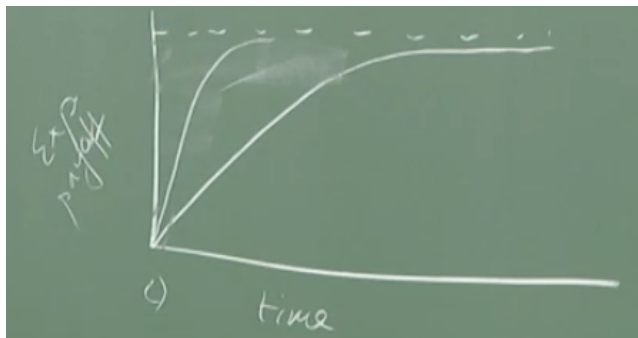


Pay off will eventually reach there

Dotted if I know it in beginning

- The gap is called - regret - wish I had known earlier

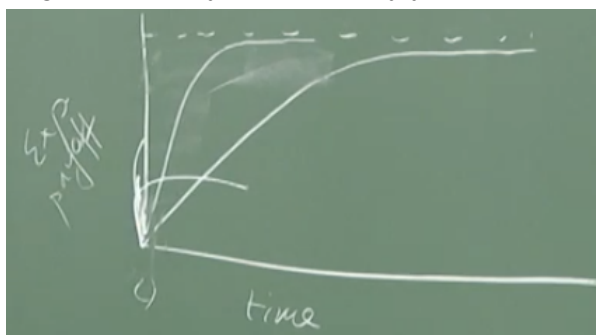
If the slope is steep - regret area will come down



You will give some amount of exploration - you are very optimistic wrt to regret

- Will miss out on key exploration
- So may not reach optimality

Regret optimality - how steeply you learn at the offset

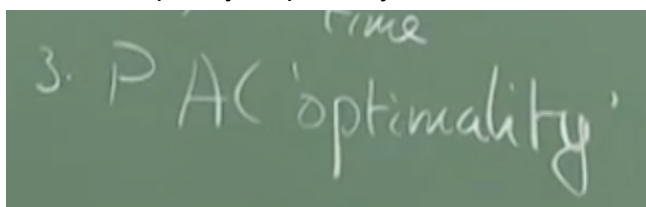


I can't be really bad not the way shown - thick small steep curve

It has to grow atleast - $\log t$ - $\log t$ is lower bound

- Area which will grow below curve will be at the rate of $\log t$

3. PAC complexity / Optimality



Probably Approximately Correct

Probably right $\Rightarrow$ right or wrong with some probability

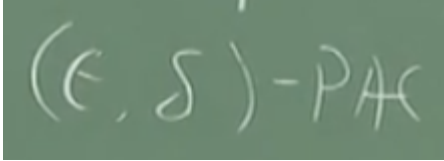
Approximately - surely not right but close to being right

- In bandit setting - i give you an arm will be close to payoff in best possible
- $q \cdot a$ will be very close to $q \cdot a^*$ (a^* is best arm)

Probably Approximately correct path:

- Very high prob Either approx correct
- Small prob away some distance from

13min


$$(\epsilon, \delta) - \text{PAC}$$

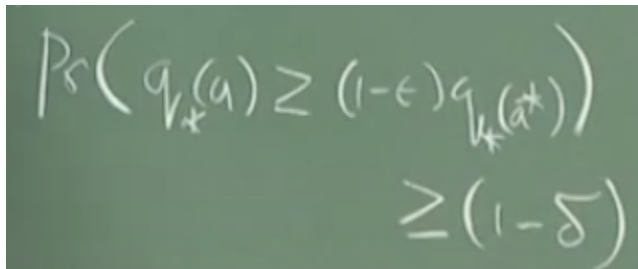
Epsilon Delta PAC

Epsilon - Approximately part

Delta - probably part

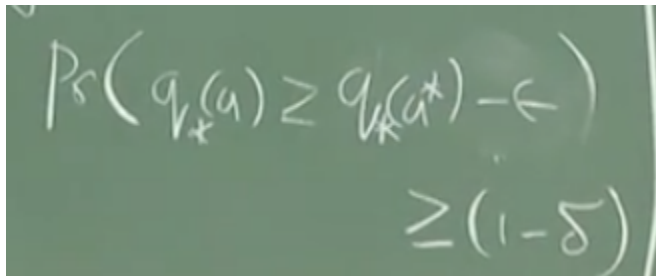
Solution $\text{prob}(1 - \delta)$ will be within epsilon of best arm

Relative guarantee - not by PAC


$$\Pr(q_{I^*}(a) \geq (1-\epsilon)q_{I^*}(a^*)) \geq (1-\delta)$$

$q^*(a)$ is arm returned to you

PAC optimality gives **absolute** guarantee (below)


$$\Pr(q_{I^*}(a) \geq q_{I^*}(a^*) - \epsilon) \geq (1-\delta)$$

It is PAC optimal

- If allow to draw infinite samples $\rightarrow$ always guarantee $\rightarrow$ now it is satisfied

Optimally comes when you want to minimize the sample complexity

- Given ϵ and δ
- Smallest no. of time pull arm to give PAC arm guarantee

Sample complexity question

Just telling what Solution complexity

- To show guarantee is complex
- Algorithms are simple

w1 L9 - Value Function Based Methods:

I don't know the q^* but will form estimate of q^* (expected payoff - do ten times, take average)

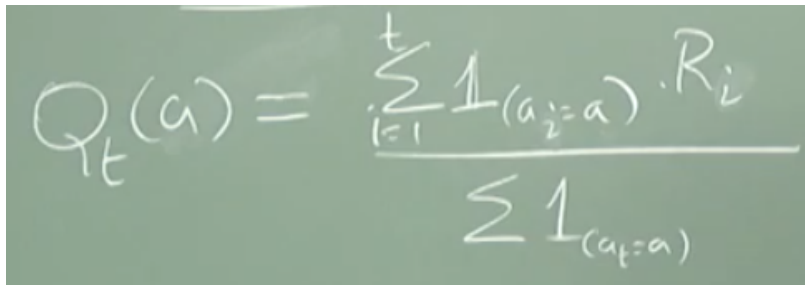
Estimate: attach estimate to an action Q_t

Attache estimate of a payoff to a state V

3 states of notations -> value function J , Q denotes cost function

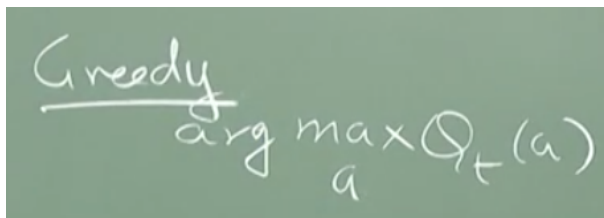
$Q_t(a)$ whenever took action a , got reward - divide by no. of times experiment

Indicator function I or delta I am using 1 (at: a)


$$Q_t(a) = \frac{\sum_{i=1}^t 1_{(a_i=a)} \cdot R_i}{\sum 1_{(a_t=a)}}$$

This gives average

Once we have estimate of q^* , how can it be used to select action - select max - greedy


$$\text{Greedy} \arg \max_a Q_t(a)$$

What does this choice corresponds to?

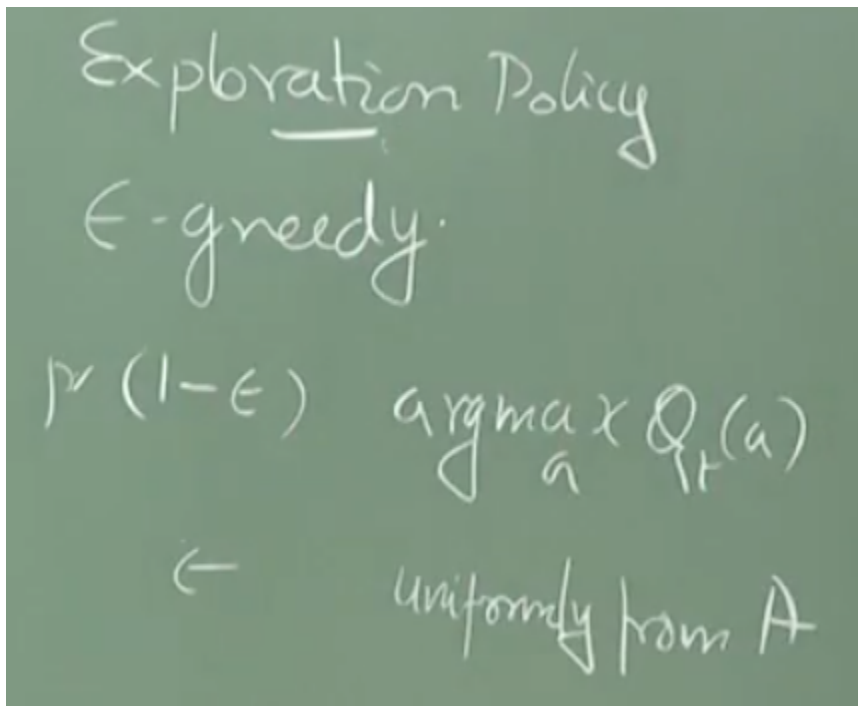
- This choice corresponds to Exploitation

Arm 3 has reward of 50 - will never ever pull arm 3 even if it has higher rewards

Exploration policy - called epsilon greedy (nothing to do with PAC epsilon)

With prob ϵ - explore

With prob $(1-\epsilon)$ will chose



uniformly from A - still have prob of $\arg\max a$

12min: Prob of taking $\arg \max$ $(1-\epsilon) + \epsilon/n$
 $(n-1) \epsilon/n$

Prob of exploration is not ϵ

By far the most popular strategy

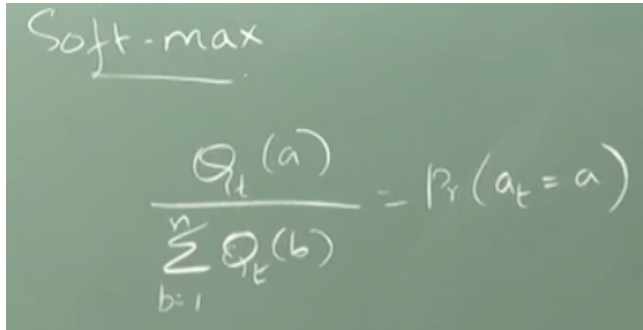
2. Soft-max exploration policy:

Ep-greedy - best according to current estimate (not q^* - it is according to Q)

Greedy bulk of prob - other equally exploration

- Don't want to pick bad, want to reduce prob for exploration
- If two actions have very small - in greedy little higher will be max
- Want to check the difference and see which one is better
- Instead of giving prob to one action
- Give prob proportional to Q

T -th step will pick action a



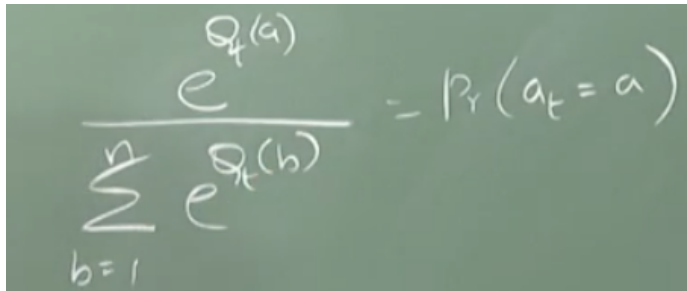
Handwritten formula for Soft-max policy:

$$\frac{Q_t(a)}{\sum_{b=1}^n Q_t(b)} = P_r(a_t = a)$$

What is the problem with them

- We can end up with -ve probability - doesn't make sense

Exponentiate



Handwritten formula for Soft-max policy using exponentiation:

$$\frac{e^{Q_t(a)}}{\sum_{b=1}^n e^{Q_t(b)}} = P_r(a_t = a)$$

Why is it called soft-max - doesn't return max - but most often

A1- 19, a2 1.2, a3 .7, a4 -0.3

Most often will return a1 to you

$$\frac{e^{Q_t(a)/\beta}}{\sum_{b=1}^n e^{Q_t(b)/\beta}} = P_t(a_t = a)$$

Include temperature para Beta

If beta is very very high - behaving at uniformly random - regardless value of Q, say if it is million, 12 or 1.7 will not matter - very hot

Make beta becomes closer to zero

- Beta = 0, it becomes max
- How sharp prob need to fall

$$P(1-\epsilon) \argmax_a Q_t(a) \equiv a^*$$

← uniformly from A

$$P(a_t = a^*) = 1 - \epsilon + \epsilon/n$$

19min:

a^* is greedy action according to current estimate

Ep-greedy and soft-max are used in FULL RL for exploration

Ep-greedy will give asymptomatic correctness

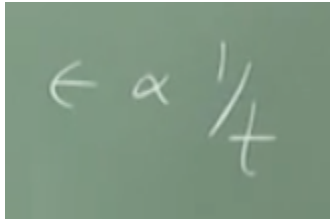
- Eventually you will converge to true expectations

If i keep ep fixed at 0.1 - 10% time i choose random

- I will know q^* , will still behaving randomly
- To achieve it cool ep
- Can not cool it too fast
- You can balance

Question: w1p1-5:

True: Cooling down ϵ too fast is problematic as it cannot guarantee correctness in value estimates


$$\epsilon \propto 1/t$$

If ϵ falls as $1/t$ it is sufficient rate - it will slow down convergence

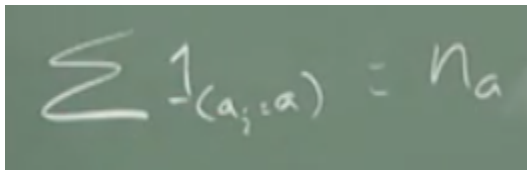
You can cool it in steps

- High - run for a while
- Bring down - run for a while etc

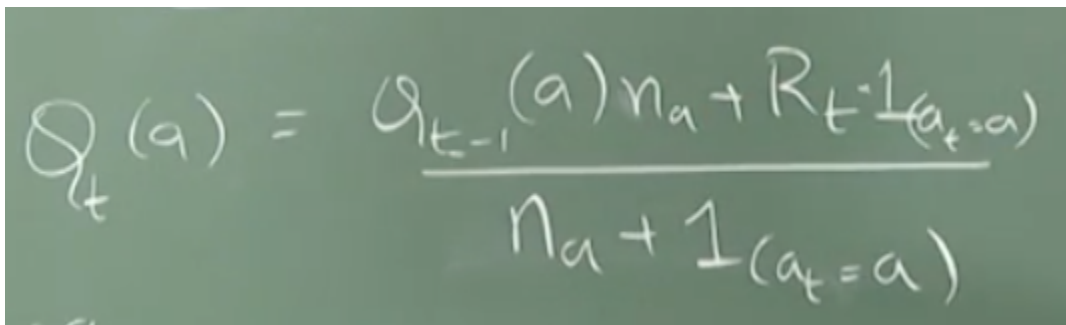
What Q_t is - look back - what were all the actions i took and what rewards i got

Is there a better way to do it?

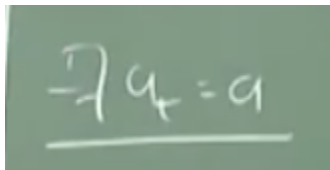
I can do Running sum


$$\sum 1_{(a_i=a)} = n_a$$

no. of times action a is taken is n_a


$$Q_t(a) = \frac{Q_{t-1}(a)n_a + R_t 1_{(a_t=a)}}{n_a + 1_{(a_t=a)}}$$

Assuming took action a , What will happen


$$\neg a_t = a$$

If $a_t = a$

$$= \frac{Q_{t-1}(a) \cdot n_a}{n_a + 1} + \frac{R_t}{n_a + 1}$$

Now i can do it in an incremental fashion

I don't need to remember

I juist need to remember

How many times i have taken action a so far and What is the previous estimate of Q_t

If $a_t = a$

$$Q_t(a) = \frac{Q_{t-1}(a) n_a + R_{t-1}(a_t=a)}{n_a + 1(a_t=a)}$$

Will get Q_t

Different way to write

$$\begin{aligned} &= \frac{Q_{t-1}(a) \cdot n_a}{n_a + 1} + \frac{R_t}{n_a + 1} \\ &= Q_{t-1}(a) \left(1 - \frac{1}{n_a + 1}\right) + \frac{R_t}{n_a + 1} \end{aligned}$$

$n_a / (n_a + 1)$ is $(1 - 1/n_a)$

Get better looking expression

$$\begin{aligned}
 &= \frac{Q_{t-1}(a) \cdot n_a}{n_a + 1} + \frac{R_t}{n_a + 1} \\
 &= Q_{t-1}(a) \left(1 - \frac{1}{n_a + 1}\right) + \frac{R_t}{n_a + 1} \\
 &= Q_{t-1}(a) + \frac{1}{n_a + 1} (R_t - Q_{t-1}(a))
 \end{aligned}$$

This is for
Stochastic averaging equation

Have old estimate Q_{t-1} (add error term target R_t - actual out put Q_{t-1})
 $1/n_a + 1$ is some kind of step size

To old estimate $\rightarrow$ adding step size times (alpha) * error term

What my real target is $\rightarrow$ expected value of R_t

- Don't know expected value of R_t , we are using some kind of
- Sample according to R_t - operate with samples
- Form estimate using samples not expected values

Choice of alpha gives exactly same value - as storing all values and taking average

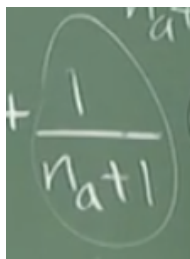
So far we have assumed - coins are same

- Distribution is stationary (not constant) at this time $\rightarrow$ distribution is always the same
- If head wears $\rightarrow$ will tend to come head earlier

What do you do in such cases

- Need a way to track changes over time

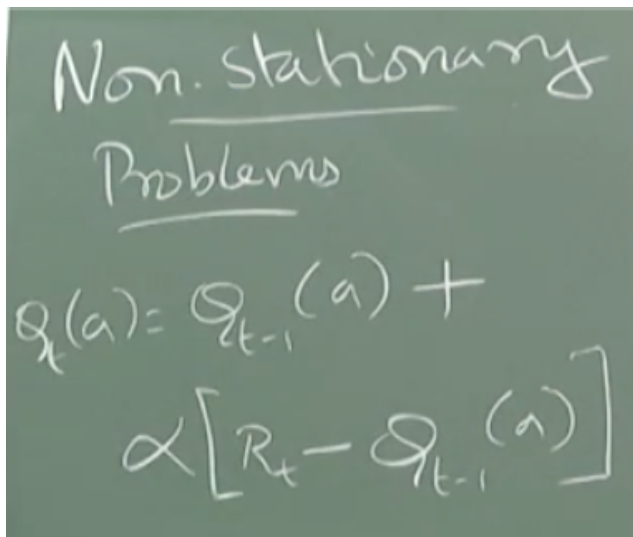
After pulling arm 1000 times



$$+ \frac{1}{n_a + 1}$$

next time the impact will be very small - for stationary sample it is ok
 It will be $1/(1000+1)$

For changing distribution - **recent** samples are more **important** to me
Older rewards will get value less
Pay attention to newer



Non-stationary
Problems

$$Q_t(a) = Q_{t-1}(a) + \alpha [R_t - Q_{t-1}(a)]$$

Instead of $1/n$ made constant $\rightarrow$ you can tune value

Problem of keeping alpha fixed

- If it is stationary you will not converge
- You will oscillating around true

In theory - alpha decay

In practice - constant alpha

Decaying alpha - changes will become small very fast